



清华大学电子工程系  
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

# 数字系统设计

## 15-神经网络训练加速

贾弘洋 & 刘勇攀

2025/5/27





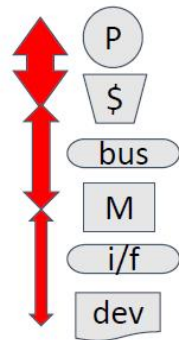
- **End-to-End Deployment**
  - **AI Processing Pipeline: pre/post ML processing**
  - **Practical Constraints: end-to-end consideration**  
**Thermal, Performance Variability, Interference**
- **Accelerator-Level Parallelism (ALP)**
  - **X-level Parallelism: BLP/ILP/TLP/DLP**
  - **ALP in Mobile SoC: diversity**
  - **Open Challenges:**  
**programmability, concurrency, communication, DSE**

# X-Level Parallelism in Computer Architecture



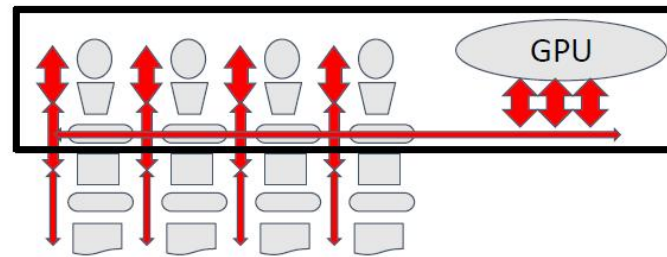
清华大学电子工程系  
Department of Electronic Engineering, Tsinghua University

- Accelerator-level parallelism (ALP): the parallelism of different workload components concurrently executing on multiple accelerators.



**1 CPU**

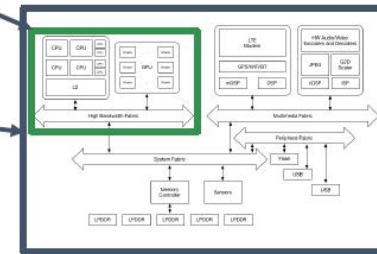
BLP+ILP  
Bit/Instrn-Level  
Parallelism



**Multicore + Integrated GPU**

+ TLP  
Thread-Level  
Parallelism

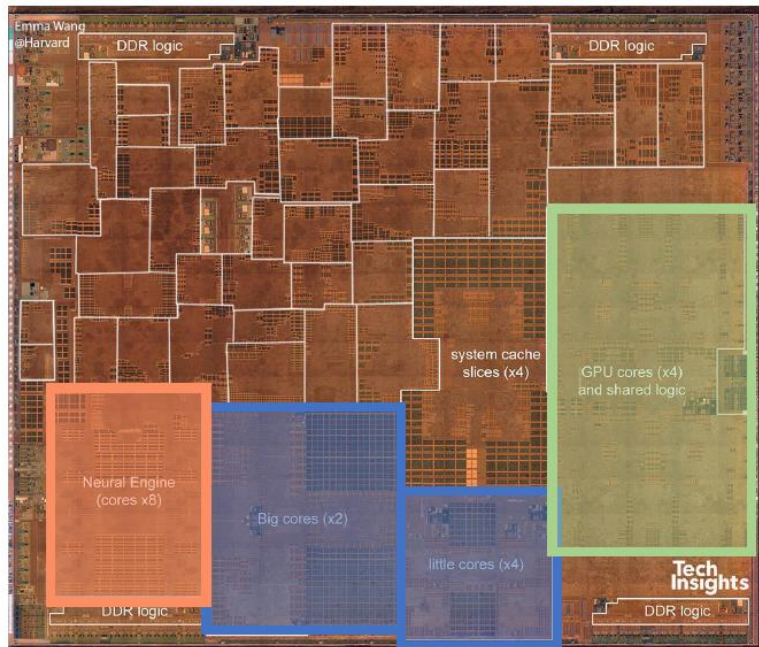
+ DLP  
Data-Level  
Parallelism



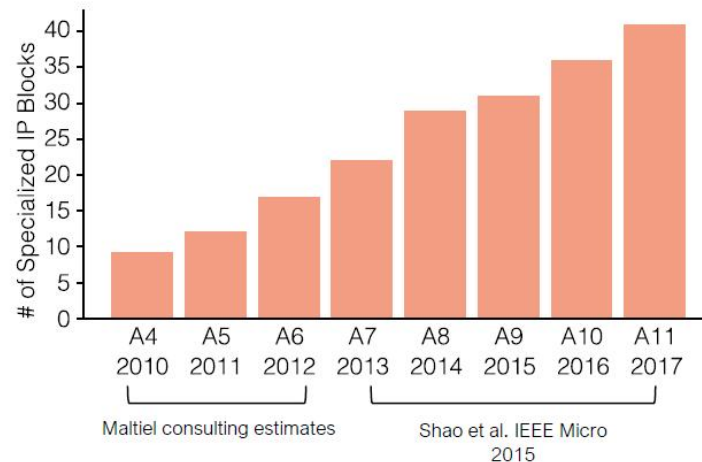
**System on a Chip (SoC)**

+ ALP  
**Accelerator-Level  
Parallelism**

# /CPU, GPU, and Accelerators



Apple A12 SoC

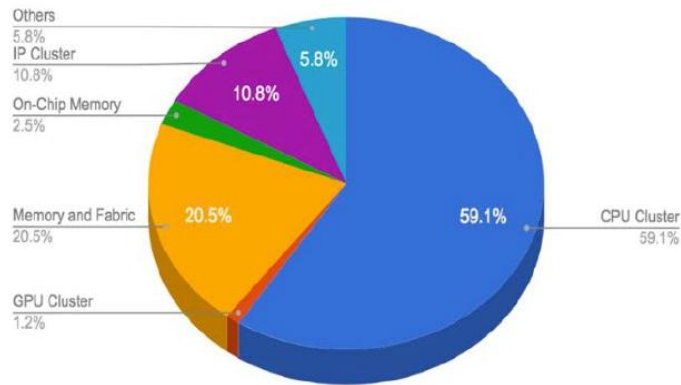


<http://vlsiarch.eecs.harvard.edu/research/accelerators/die-photo-analysis/>

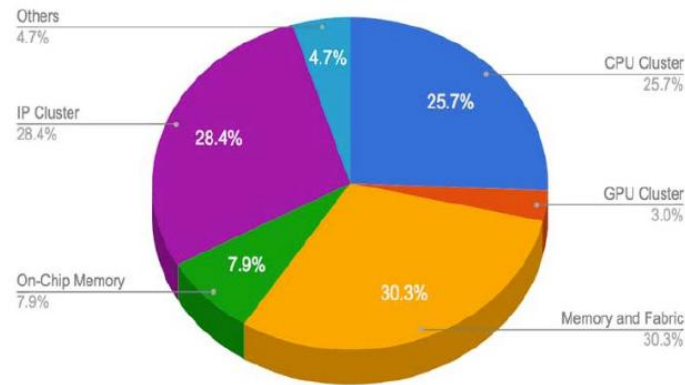
# / Mobile SoC Usecase



- Different usecases have different power profiles.



Web browser scrolling  
(total 651.77 mW)



4K video recording  
(total 3,473.55mW)

*Two Billion Devices and Counting: An Industry Perspective on the State of Mobile Computer Architecture, IEEE Micro'2018*



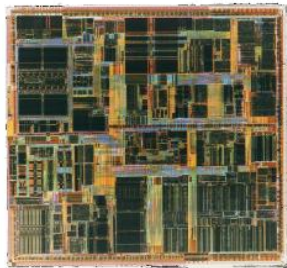
# / X-level Parallel Hardware + Software (Model)

Single-threaded  
languages,  
compilers,  
runtimes, etc.  
**hides** parallelism

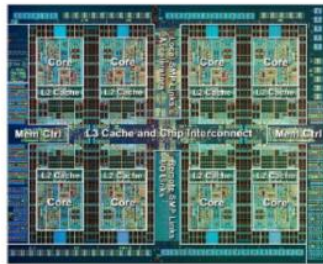
Software  
**abstracts** TLP,  
e.g., pThreads,  
OpenMP, & MPI.  
Also **hidden** in  
cloud, etc.

Software **abstracts**  
TLP+DLP, e.g.,  
CUDA, OpenCL, &  
graphics OpenGL  
Also **hidden** in  
cloud, etc.

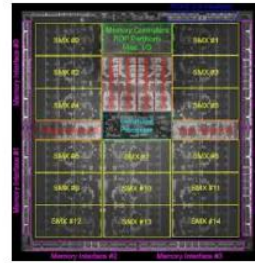
Local software stack  
**abstracts** each  
accelerator.  
**But no good, general  
software abstraction  
for SoC ALP!**



Intel Pentium Pro  
BLP+ILP



IBM Power 7  
BLP+ILP+TLP



Nvidia GK110  
BLP+TLP+DLP



Apple A12  
BLP+ILP+TLP+DLP+ALP



# / Overview

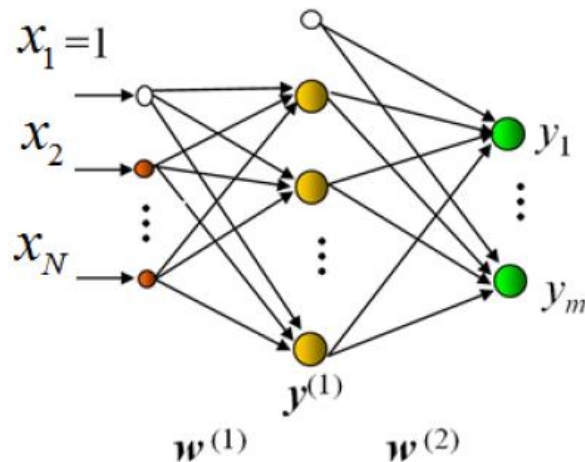
- **Training Flow**
- **Optimizer**
- **Training Accelerator**

# / Training Flow

- Back Propagation (BP)学习算法

- 神经网络的有监督学习算法
- BP学习算法由正向传播和误差反向传播组成：

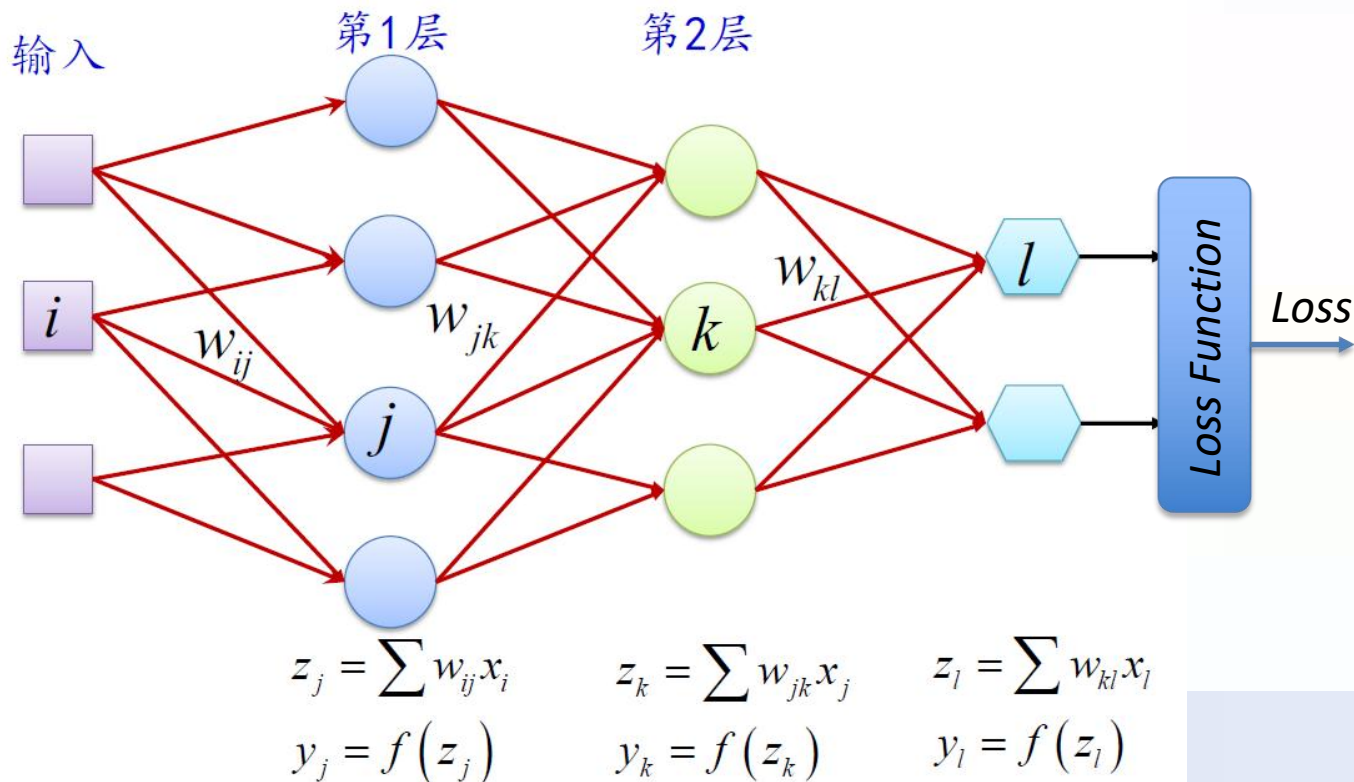
- ① 正向传播是输入信号从输入层经隐含层，传向输出层，若输出层得到了期望的输出，则学习算法结束；否则，转至反向传播。
- ② 反向传播是将误差（样本输出与网络输出之差）按原联接通路反向计算，由梯度下降法调整各层节点的权值和阈值，使误差减小。





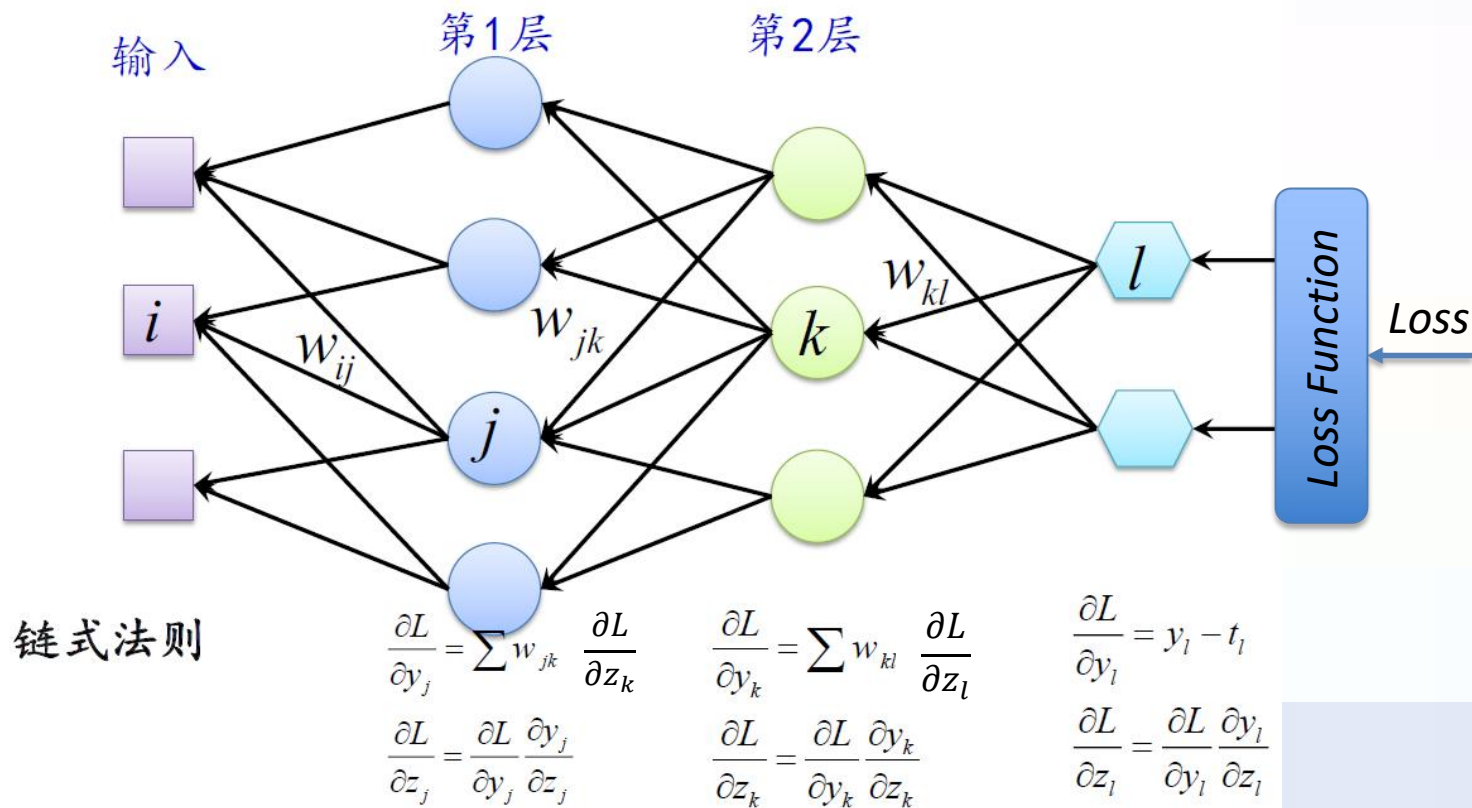
# / Training Flow

## BP学习算法：前向传播



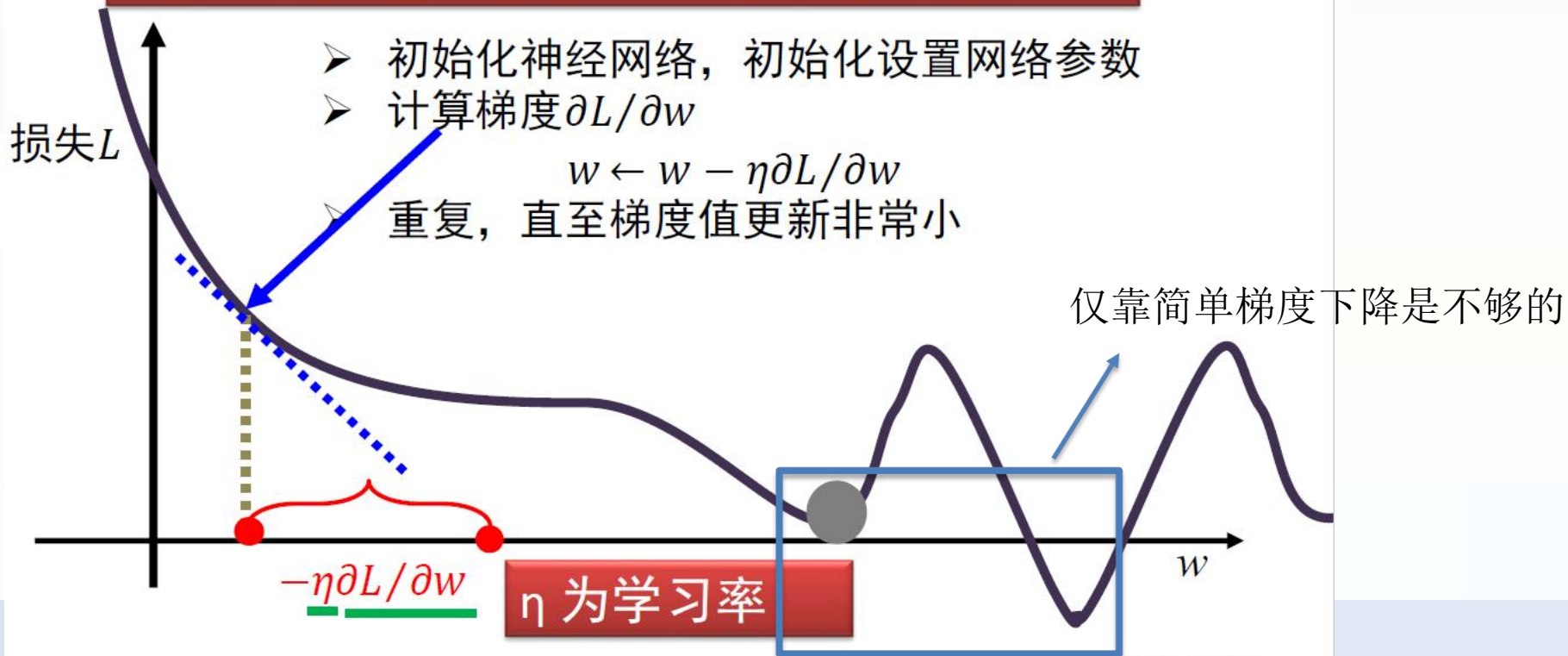
# / Training Flow

## BP学习算法：误差反向传播



# / Training Flow

学习获取使损失函数最小化的网络参数  $\theta^*$



# / Training Flow

## ❑ Batch Gradient Descent (BGD)

- ✓ 利用所有样本计算梯度

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

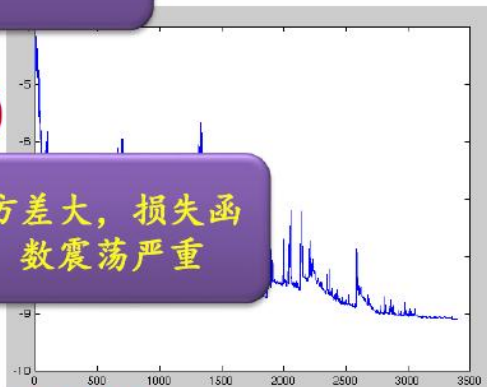
速度慢，数据量大时内存不足

## ❑ Stochastic Gradient Descent (SGD)

- ✓ 随机选取单个样本计算梯度

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta, x_i, y_i)$$

方差大，损失函数震荡严重



## ❑ Mini-batch Stochastic Gradient Descent

- ✓ 随机选取mini-batch样本计算梯度

$$\theta \leftarrow \theta - \eta \left[ \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L(\theta, x_i, y_i) \right]$$

## /Optimizer

- 避免陷入局部最优的问题：动量
- 避免陷入过拟合：衰减系数
- 优化器的一般范式：（带动量和衰减系数的optimizer）

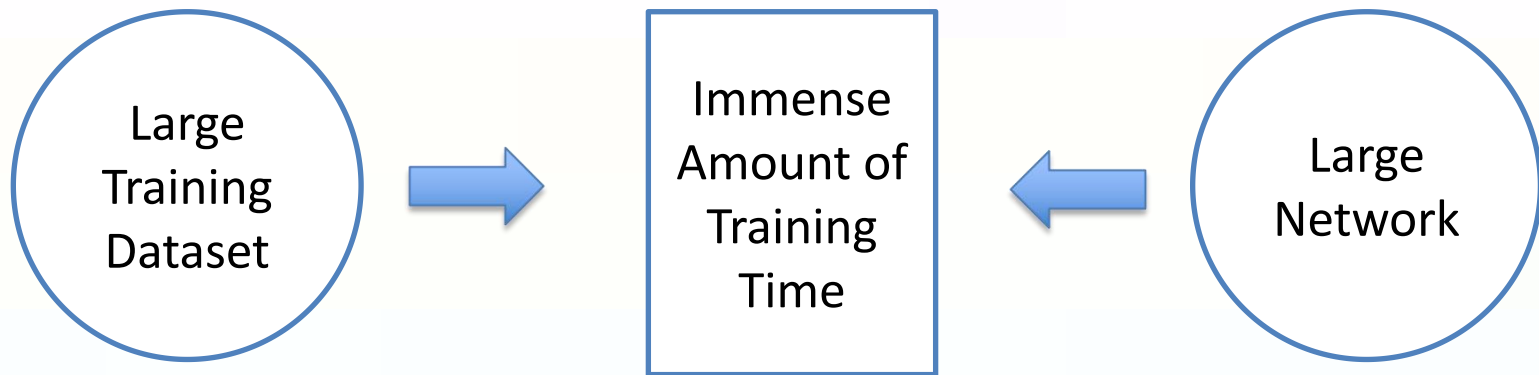
$$\begin{aligned} v_{i+1} &:= \overset{\text{Momentum}}{0.9 \cdot v_i} - \overset{\text{Weight Decay}}{0.0005 \cdot \epsilon \cdot w_i} - \epsilon \cdot \left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i} \\ w_{i+1} &:= w_i + v_{i+1} \end{aligned}$$

Learning rate

Gradient of cost function

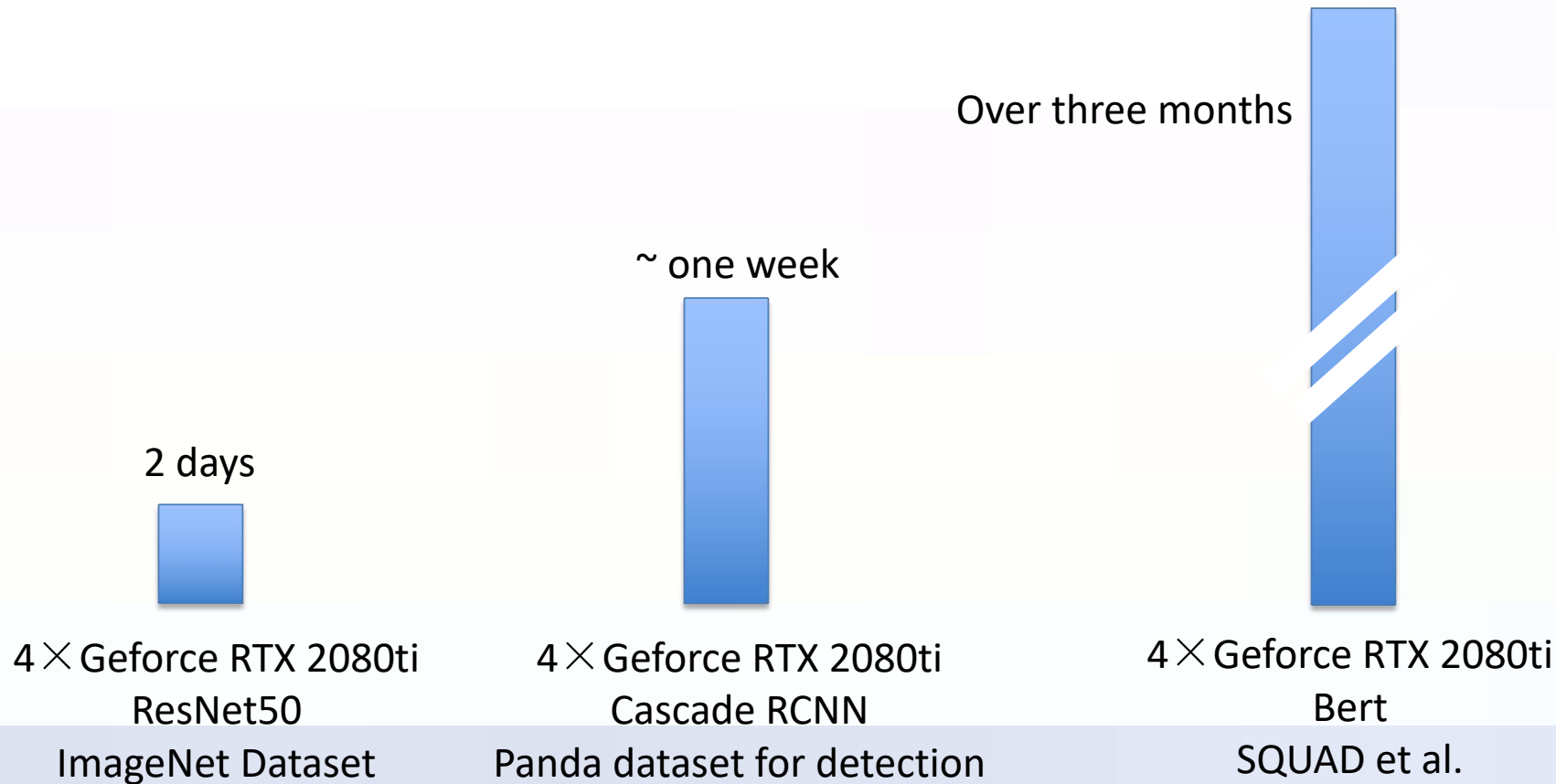
## /Training

- 带动量和衰减系数的SGD优化器，在很多经典任务的训练上，都有很好的效果。
- 还遗留了什么**问题**？





# / Training



# /Training

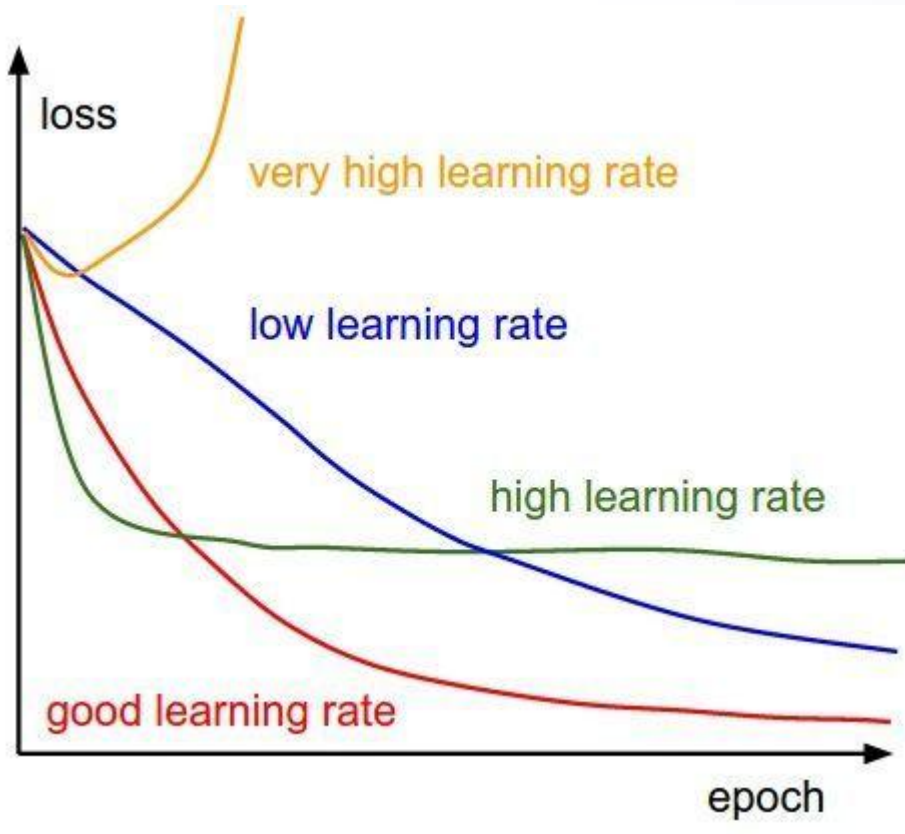
- Accelerate the training Process is a crucial problem
  - Design a better training scheme
  - Design a better optimizer
  - Design a better training processor

# / Overview

- **Training Flow**
- **Optimizer**
- **Training Accelerator**

## / Training Scheme

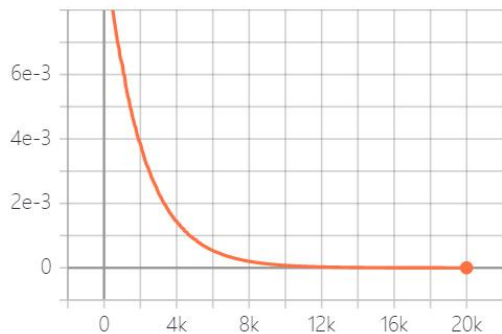
- 学习率直接影响收敛速度和训练时长。
- 在训练初期，使用较大学习率，加快收敛。
- 训练一段时间后，选择较小学习率，使得能够准确逼近最优解。



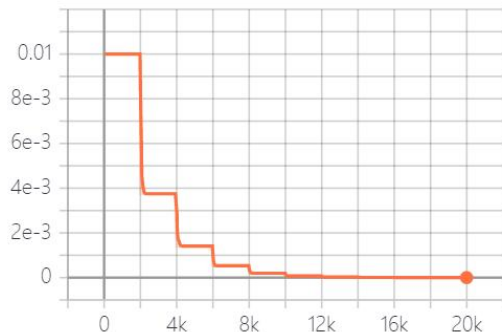
# Training Scheme

- 学习率衰减策略（指数、线性、阶梯、余弦等等）

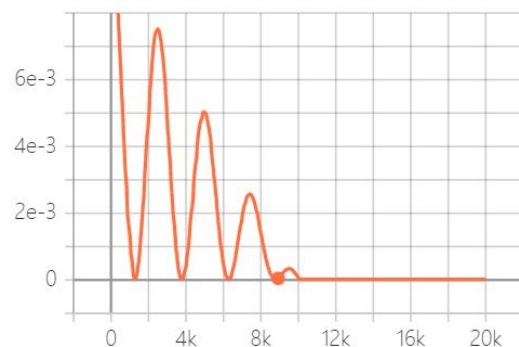
natural\_exp\_decay\_no\_stair\_1



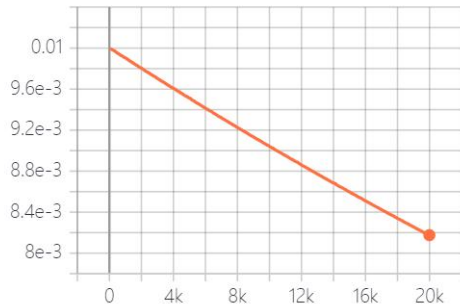
natural\_exp\_decay\_use\_stair\_1



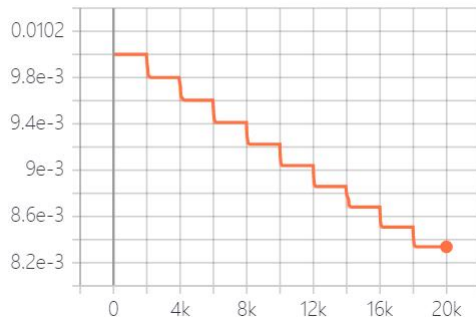
linear\_cosine\_decay\_4\_1



linear\_decay\_no\_stair\_1



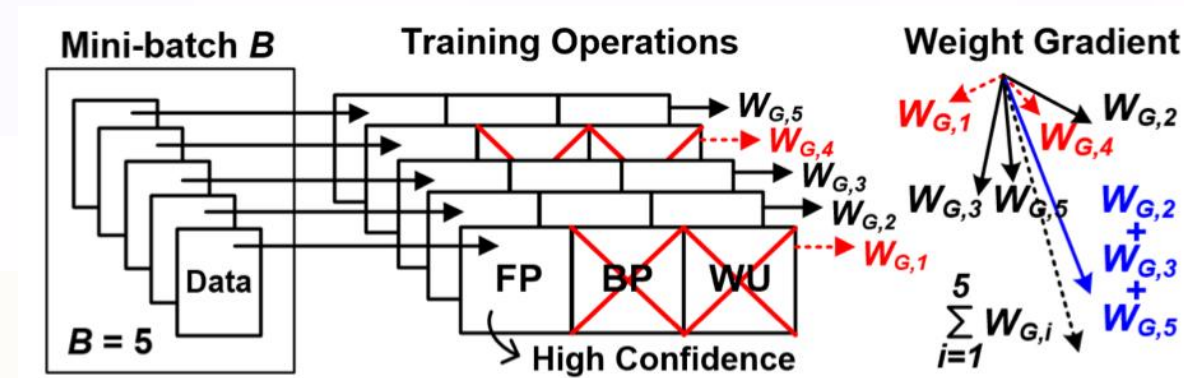
linear\_decay\_use\_stair\_1



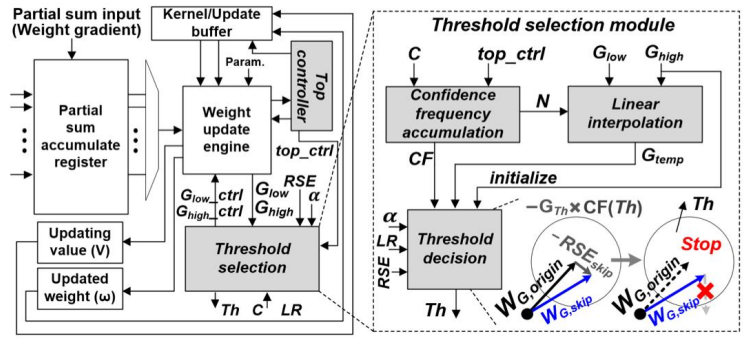
- 不同的任务，有不同的适配衰减方法。

# Training Scheme

- 训练图像数量的多寡也是影响训练时长的重要因素



DAC, 2020



- 主要思想：简单训练数据很早就能收敛。在训练中后期，无需重复训练简单数据。
- 效果：CIFAR10数据集上，可以跳过98%的图像，不损失精度。ImageNet可以跳过42%的图像，损失0.6%精度。



# / Overview

- **Training Flow**
- **Optimizer**
- **Training Accelerator**

## /Optimizer

- 优化器，其设计目的是指引目标函数的各个参数，往**正确的方向更新合适的大小**，使得更新后的参数让损失函数值不断逼近全局最小。
- 一般步骤
  - 计算梯度
  - 计算更新方向及步长
  - 更新参数
- 好的优化器，能够加速收敛，缩短训练时间。

# /Optimizer

- 避免陷入局部最优的问题：动量
- 避免陷入过拟合：衰减系数
- 优化器的一般范式：（带动量和衰减系数的optimizer）

$$\begin{aligned} v_{i+1} &:= \overset{\text{Momentum}}{\downarrow} 0.9 \cdot v_i - \overset{\text{Weight Decay}}{\downarrow} 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i} \\ w_{i+1} &:= w_i + v_{i+1} \end{aligned}$$

Learning rate

Gradient of cost function

- 除了SGD外，还有什么优化器？

## /Nesterov Optimizer

- 更相信之前的动量方向。
- 当前更新时，更新位置为沿着之前动量方向往前走一小步的位置。
- $\theta$ 表示权重，收敛速度比基础momentum更快。

$$d_i = \beta d_{i-1} + g(\theta_{i-1})$$

$$\theta_i = \theta_{i-1} - \alpha d_i$$

Previous momentum

$$d_i = \beta d_{i-1} + g(\theta_{i-1} - \alpha \beta d_{i-1})$$

$$\theta_i = \theta_{i-1} - \alpha d_i$$

This one

## / AdaGrad Optimizer

- 给梯度做一个自适应的衰减系数，系数计算方式为历史梯度的平均
- 适用于模型后期的训练，加快收敛。
- 适用于训练数据集类别不平衡的情况 (g表示梯度)

$$W_{t+1} = W_t - \frac{\eta_0}{\sqrt{\sum_{t'=1}^t (g_{t',i})^2} + \epsilon} \odot g_{t,i}$$

## / AdaDelta Optimizer

- 本次更新方向与历史方向均相关。
- 可以加速模型中期的训练速度，但在后期会在一个good point附近抖动。
- $g$ 为梯度， $E[]$ 表示求平均， $W$ 为权重更新量。

$$\begin{cases} E[g^2]_t = \alpha E[g^2]_{t-1} + (1 - \alpha)g_t^2 \\ \Delta W_t = -\frac{\sqrt{\sum_{i=1}^{t-1} \Delta W_i}}{\sqrt{E[g^2]_t + \epsilon}} \\ W_{t+1} = W_t + \Delta W_t \end{cases}$$

## /Adam Optimizer (ICLR-2014)

- 非常鲁棒，但因为要计算二阶矩阵，迭代一次耗时更长。
- 只需要更少的训练次数，一般收敛更快。
- $v$ 表示一阶速度， $s$ 是二阶速度。

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) \nabla_{\theta} L(\theta_t)$$

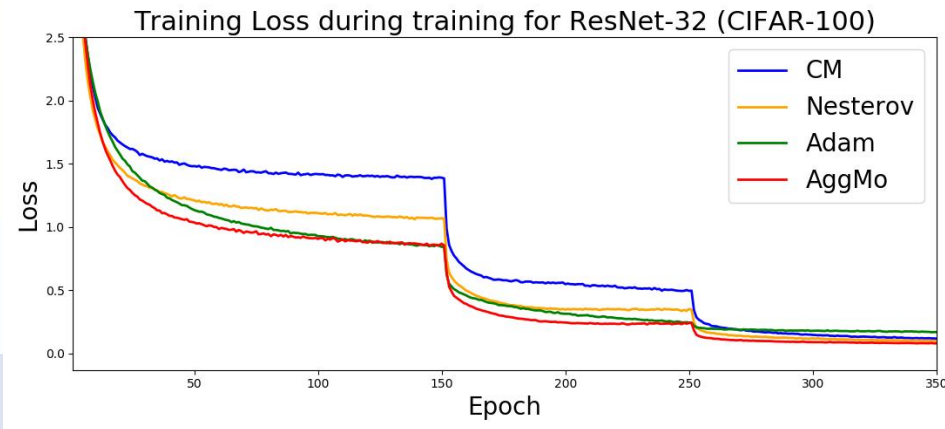
$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) (\nabla_{\theta} L(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{v_t}{\sqrt{s_t + \varepsilon}} \nabla_{\theta} L(\theta_t)$$

# / AGGMO Optimizer (ICLR-2019)

- Aggregate Momentum
- 之前的优化器只会有一个 $\beta$ 值调节动量的贡献，该方法记录了多个 $[\beta_0, \beta_1, \dots]$ 下的更新方向，做加权平均以使得当前优化的方向更加准确。

$$\mathbf{v}_t^{(i)} = \beta^{(i)} \mathbf{v}_{t-1}^{(i)} - \nabla_{\theta} f(\theta_{t-1}), \text{ for all } i,$$
$$\theta_t = \theta_{t-1} + \frac{\gamma_t}{K} \sum_{i=1}^K \mathbf{v}_t^{(i)},$$

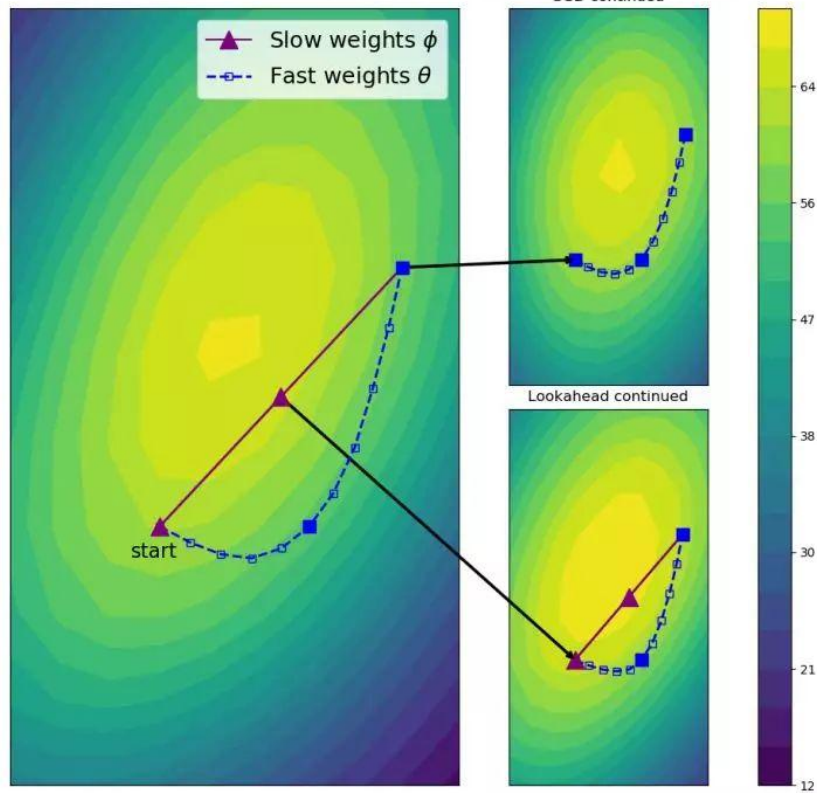




# /Lookahead Optimizer (NIPS-2019)

- 先往前看k步，找到当前更新k步后的更新方向。（fast weights）
- 这个方向被认为是更准确的方向，按照这个方向更新一次权重。（slow weights）
- Adam和SGD利用这个训练策略，都能加速收敛。

CIFAR-100 accuracy surface with Lookahead interpolation



# /Optimizer Overview

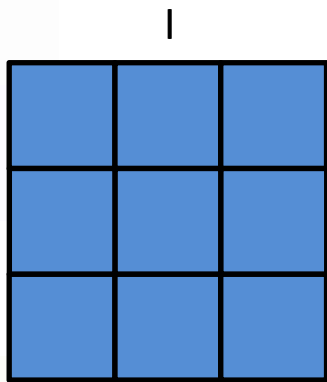
- 方法与手段
  - 找到正确的更新方向
  - 在正确的方向上，尽可能跨上最大的正确步伐
- 加速效果比较有限。由于无法预知能收敛的训练次数，难以量化实际使用时的加速效果。

# / Overview

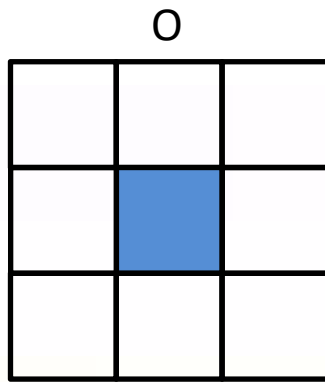
- **Training Flow**
- **Optimizer**
- **Training Accelerator**

# / Analysis

梯度回传



前向传播



$$O_{0,0} = W_{2,2} * I_{1,1} + \dots$$

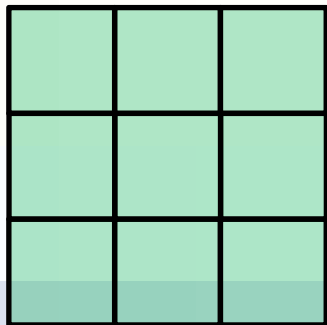
$$O_{0,1} = W_{2,1} * I_{1,1} + \dots$$

$$O_{0,2} = W_{2,0} * I_{1,1} + \dots$$

.....

$$O_{2,2} = W_{0,0} * I_{1,1} + \dots$$

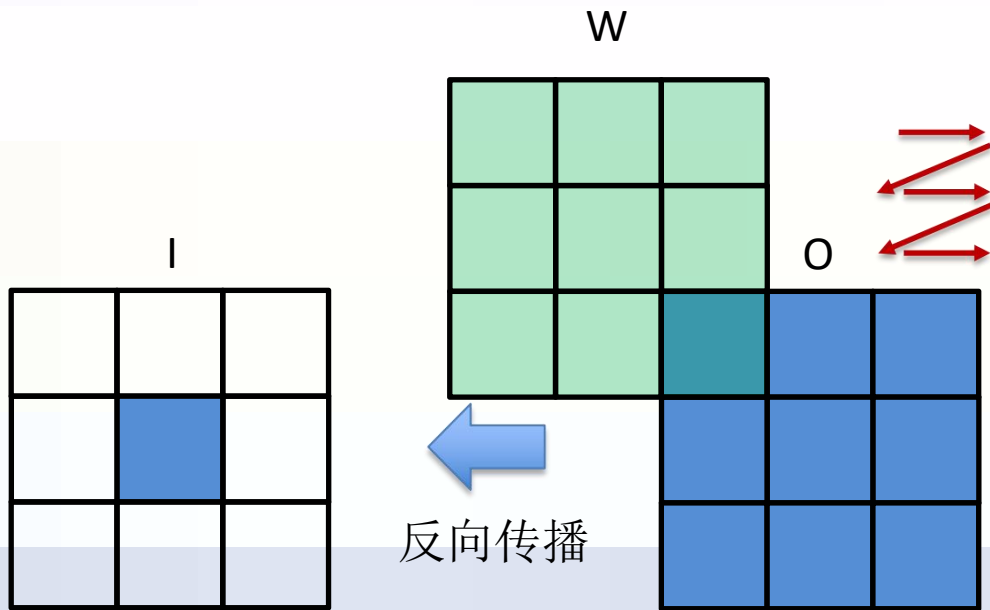
W



W 与 I 卷积：对应位置相乘再加和

# /Analysis

## 梯度回传



$$O_{0,0} = W_{2,2} * I_{1,1} + \dots$$

$$O_{0,1} = W_{2,1} * I_{1,1} + \dots$$

$$O_{0,2} = W_{2,0} * I_{1,1} + \dots$$

.....

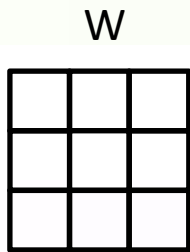
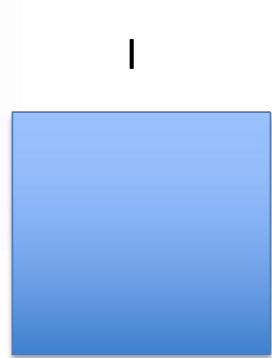
$$O_{2,2} = W_{0,0} * I_{1,1} + \dots$$



$$\frac{\partial L}{\partial I_{1,1}} = \sum \frac{\partial L}{\partial O_{a,b}} * W_{2-a,2-b} = \frac{\partial L}{\partial O} \odot W^T$$

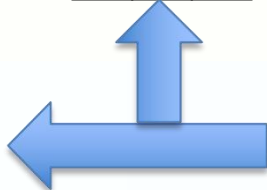
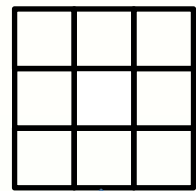
反向传播本质也是卷积过程！

# /Analysis



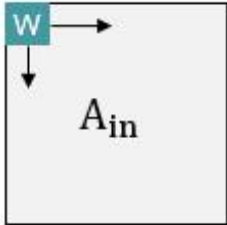
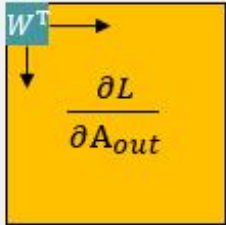
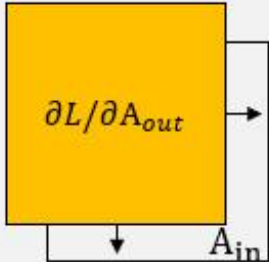
权重更新

$$O = W \odot I$$

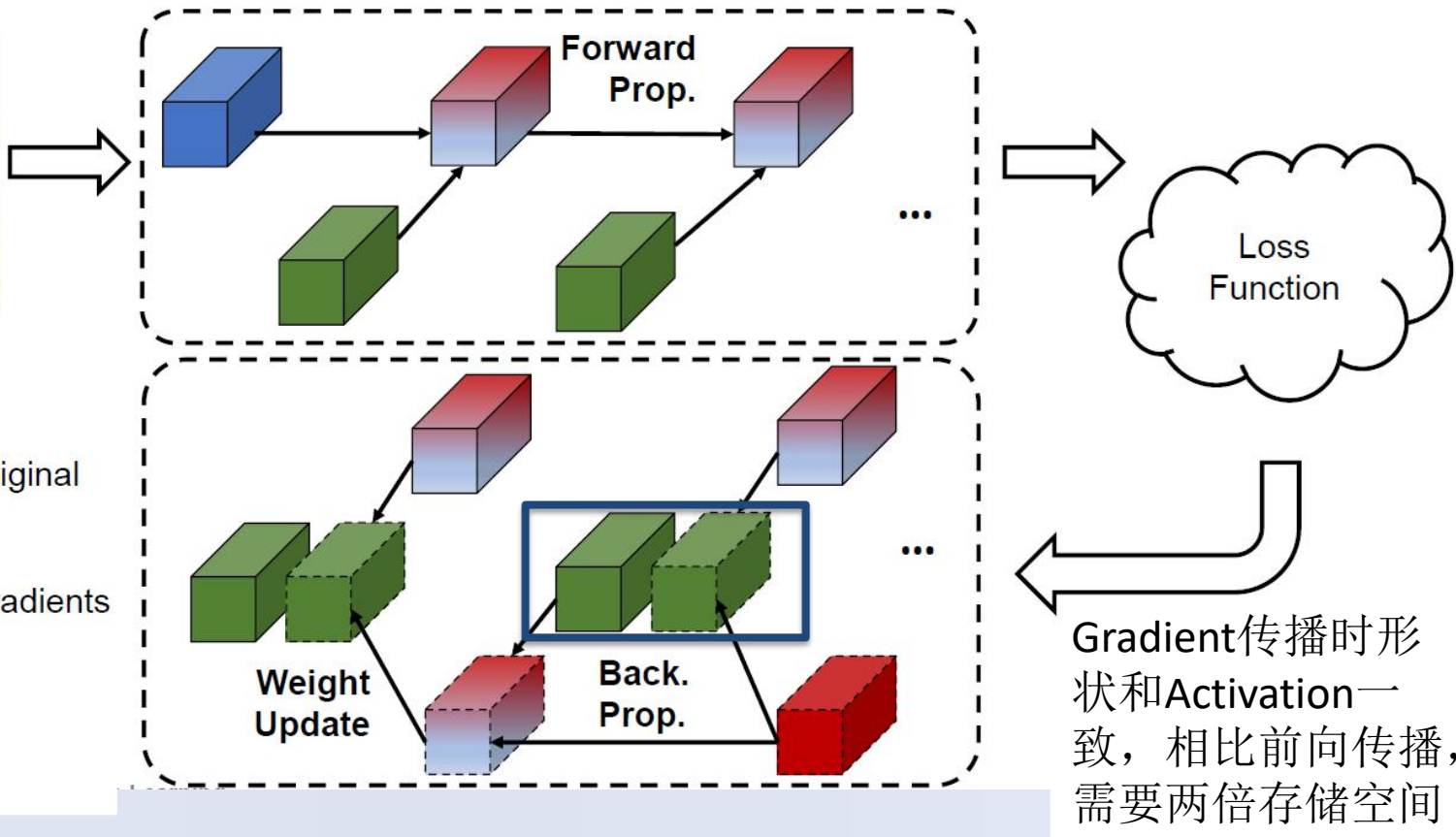


$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial O} * I$$

# / Analysis

|    | Forward                                                                                                      | Backward                                                                                                                                                                | Weight Update                                                                                                                                                                         |
|----|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 输入 | $A_{in}, W$                                                                                                  | $\frac{\partial L}{\partial A_{out}}, W^T$                                                                                                                              | $\frac{\partial L}{\partial A_{out}}, A_{in}$                                                                                                                                         |
| 输出 | $A_{out}$                                                                                                    | $\frac{\partial L}{\partial A_{in}}$                                                                                                                                    | $\frac{\partial L}{\partial W}$                                                                                                                                                       |
| 操作 | $A_{out} = A_{in} \odot W$  | $\frac{\partial L}{\partial A_{in}} = \frac{\partial L}{\partial A_{out}} \odot W^T$  | $\frac{\partial L}{\partial W} = \sum_{i=1}^{Batch} \frac{\partial L}{\partial A_{out}} * A_{in}$  |

# / Analysis





# / Analysis

- 如何设计一个更好的训练加速器？
  - 训练加速器所需的操作（卷积、矩阵逐元素乘等），本质上都能由一般的前向加速器完成。那还有哪些设计空间？
- 目标：更快、更高能效、更少内存占用。

## / Training Accelerator

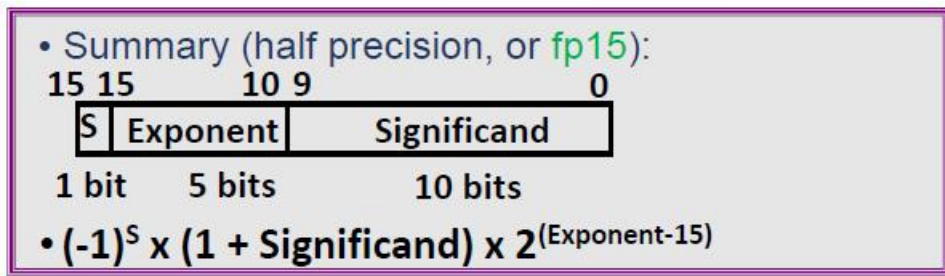
- 从推理加速器中获取灵感
  - 量化？前向加速器所用的定点量化数值分辨率太低，而梯度更新需要很高的分辨率。

| Method                 | Model     | Bit-Width (W/A/E/ACC)                                                    | Acc.        | FP baseline | Acc. Drop |
|------------------------|-----------|--------------------------------------------------------------------------|-------------|-------------|-----------|
| (Köster et al., 2017)  | AlexNet   | 16 16 16 32                                                              | 80.1%(Top5) | 79.9%(Top5) | -0.2%     |
| (Das et al., 2018)     | VGG-16    | 16 16 16 32                                                              | 68.2%       | 68.1%       | -0.1%     |
| (Das et al., 2018)     | GoogleNet | 16 16 16 32                                                              | 69.3%       | 69.3%       | 0         |
| (Banner et al., 2018a) | ResNet-18 | 8 8 16 f32                                                               | 66.4%       | 67.0%       | 0.6%      |
| (Zhou et al., 2016)    | AlexNet   | 8 8 8 32                                                                 | 53.0%       | 55.9%       | 2.9%      |
| (Yang et al., 2020)    | ResNet-18 | 8 8 8 32                                                                 | 64.8%       | 68.7%       | 3.9%      |
| (Yang et al., 2020)    | ResNet-34 | 8 8 8 32                                                                 | 67.6%       | 72.0%       | 4.4%      |
| (Wu et al., 2018)      | AlexNet   | 2 8 8 32                                                                 | 48.4%       | 56.0%       | 7.6%      |
| (Sun et al., 2019)     | ResNet-18 | $\langle 5, 3 \rangle$ $\langle 5, 3 \rangle$ $\langle 5, 3 \rangle$ f32 | 69.0%       | 69.3%       | 0.3%      |
| (Cambier et al., 2020) | ResNet-18 | $\langle 5, 2 \rangle$ $\langle 5, 2 \rangle$ $\langle 5, 2 \rangle$ f32 | 69.6%       | 70.3%       | 0.7%      |

- 解决方案：低比特浮点数训练

## / Training Accelerator

- 16 bit单精度浮点，在设计中，我们可以依据数据分布重新分配Exponent和Significand的比特分布。



- 微软在brainwave工程中，更是使用了半精度浮点数（8bit），进一步进行数据压缩。

# / Training Accelerator

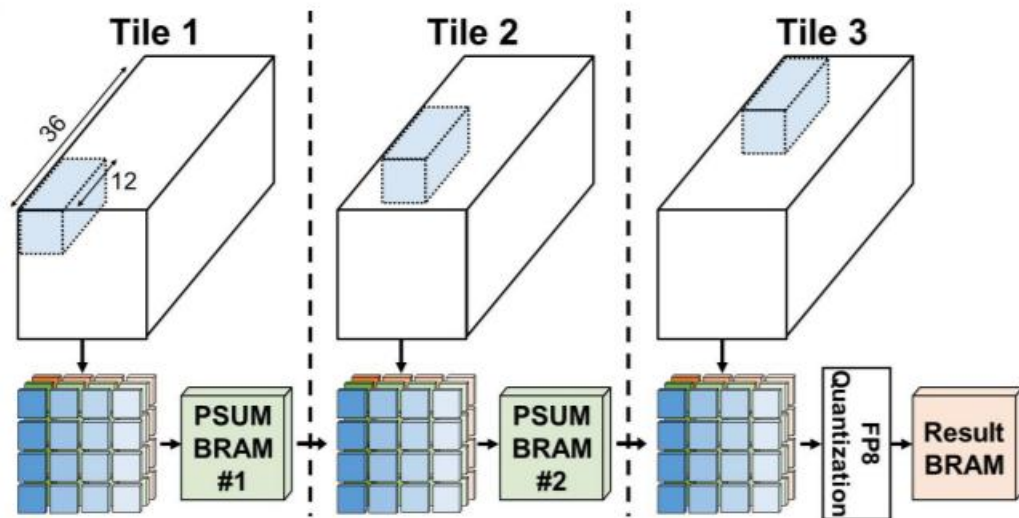
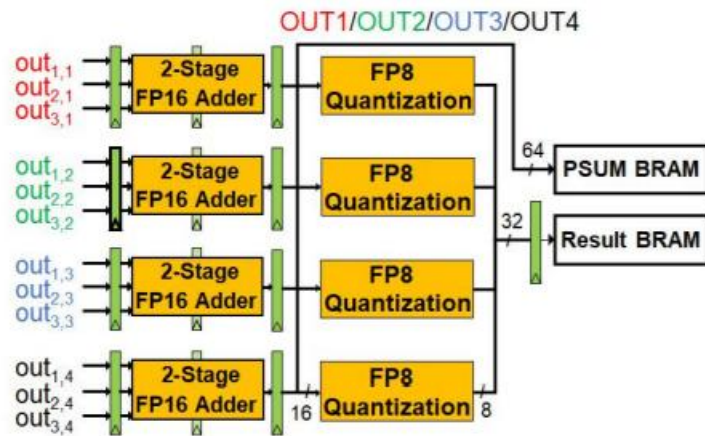
- Challenge

- 溢出错误（grad overflow / underflow）FP16（ $6E-8 \sim 65504$ ）的表示范围远小于FP32（ $1E-45 \sim 1E38$ ）。
- 舍入误差（rounding error）当梯度过小时，更新权值会出现“大吞小”现象： $2^{-3} + 2^{-14} = 2^{-3}$

# / Training Accelerator

- 解决方案1: 多累加几次, 再量化。

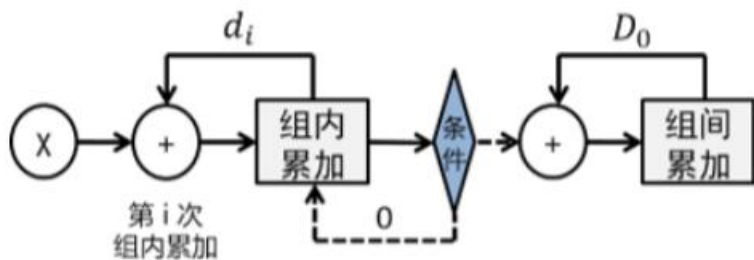
JETCAS@2021



# / Training Accelerator

- 自适应动态分组累加，满足条件即可量化。

ASPDAC@2022



$$1024 + \overbrace{(1 + \dots + 1)}^{\text{N0路分组}} + \overbrace{(32 + \dots + 4)}^{\text{N1路分组}} + (\dots)$$

N0路满足条件    N1路满足条件

# / Training Accelerator

- 解决方案2: 依概率量化

NIPS@2020

$$\text{Round}(x) = \begin{cases} s \cdot 2^e \cdot (1 + \lfloor m \rfloor + \epsilon) & \text{with probability } \frac{m - \lfloor m \rfloor}{\epsilon}, \\ s \cdot 2^e \cdot (1 + \lfloor m \rfloor) & \text{with probability } 1 - \frac{m - \lfloor m \rfloor}{\epsilon}, \end{cases}$$

- 优点: 不会改变数据分布的期望, 收敛效果好。

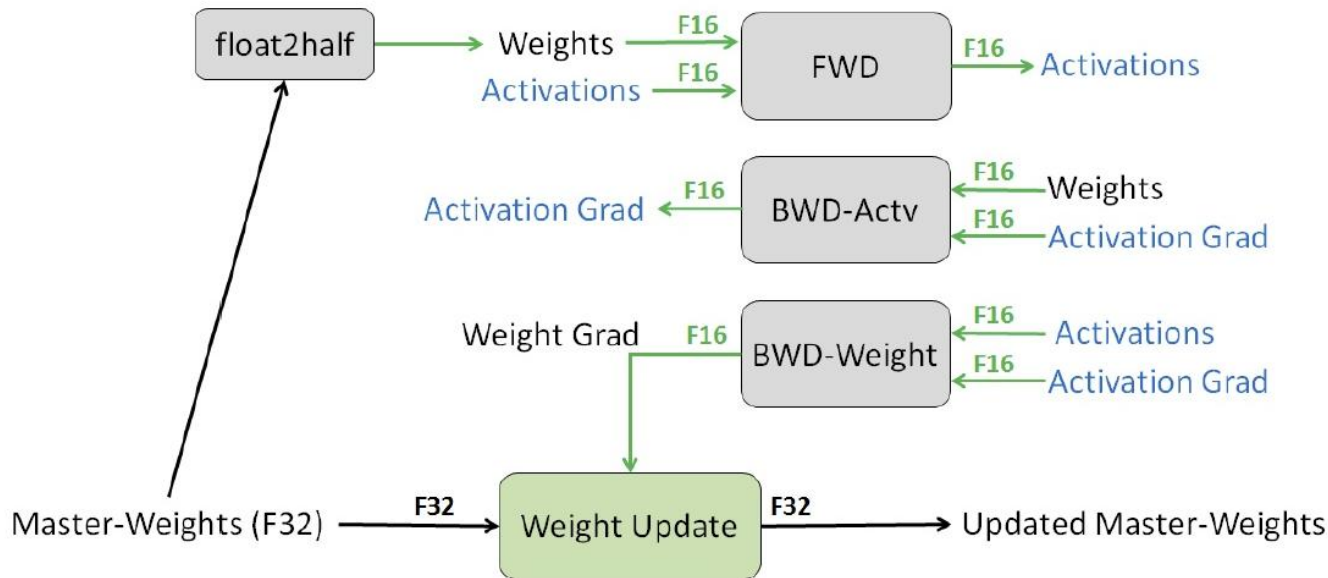
$$E(x) = E(\text{Round}(x))$$

- 缺点: 通常需要生成随机数, 纯硬件实现难度大。

# / Training Accelerator

APEX by NVIDIA

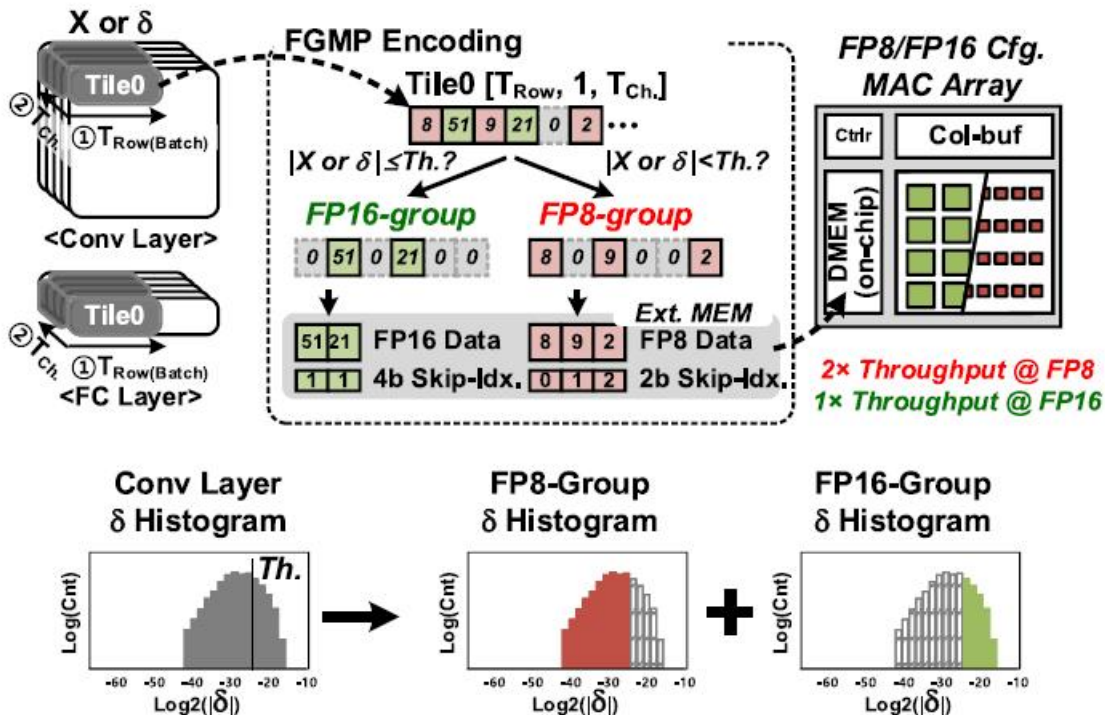
- 解决方案3：混合精度训练
- 名单制度：大部分算子使用FP16精度，Softmax等算子采用FP32。
- 动态overflow检测调整。当发生溢出时，整体放大/缩小loss。





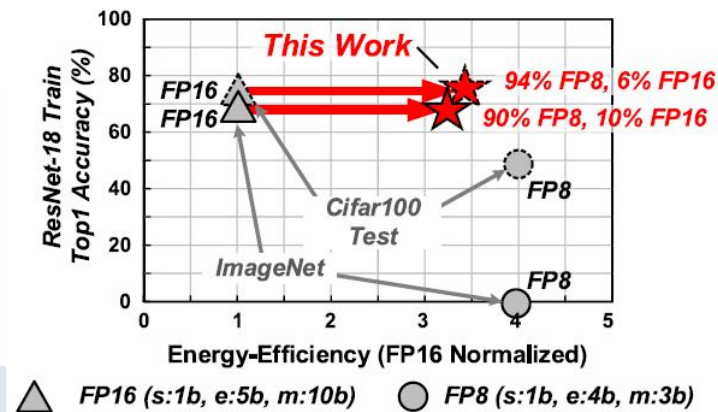
# Training Accelerator

- 解决方案3: 混合精度训练



ISCL@2019

- 按照分布确定某数使用FP8还是FP16
- FGMP: FINE-GRAINED MIXED PRECISION



# / Training Accelerator

- 解决方案3: 混合精度训练

ISCA@2021

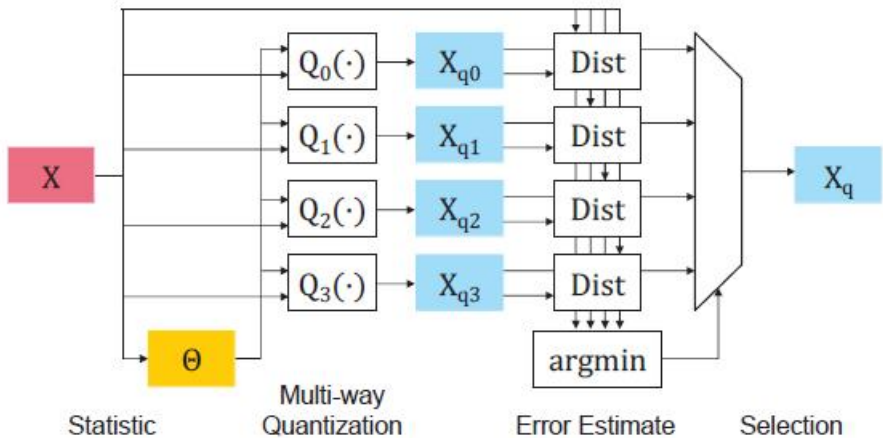


Fig. 5. Error-estimation-based Quantization Multiplexing.

- SQU: statistic quantization unit

- 不使用全局分布确定量化比特
- 按块收集统计数据，进行量化

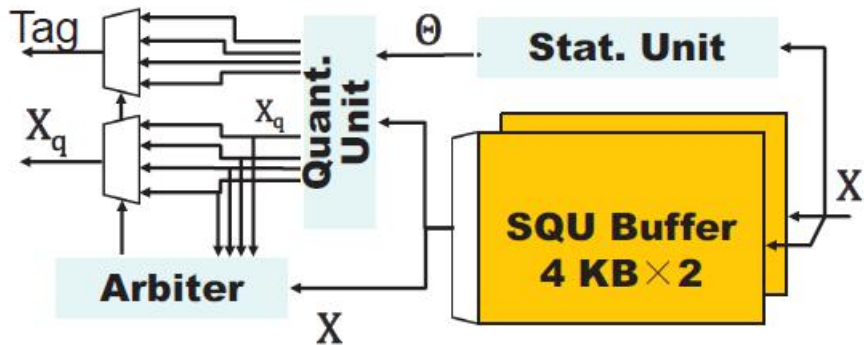
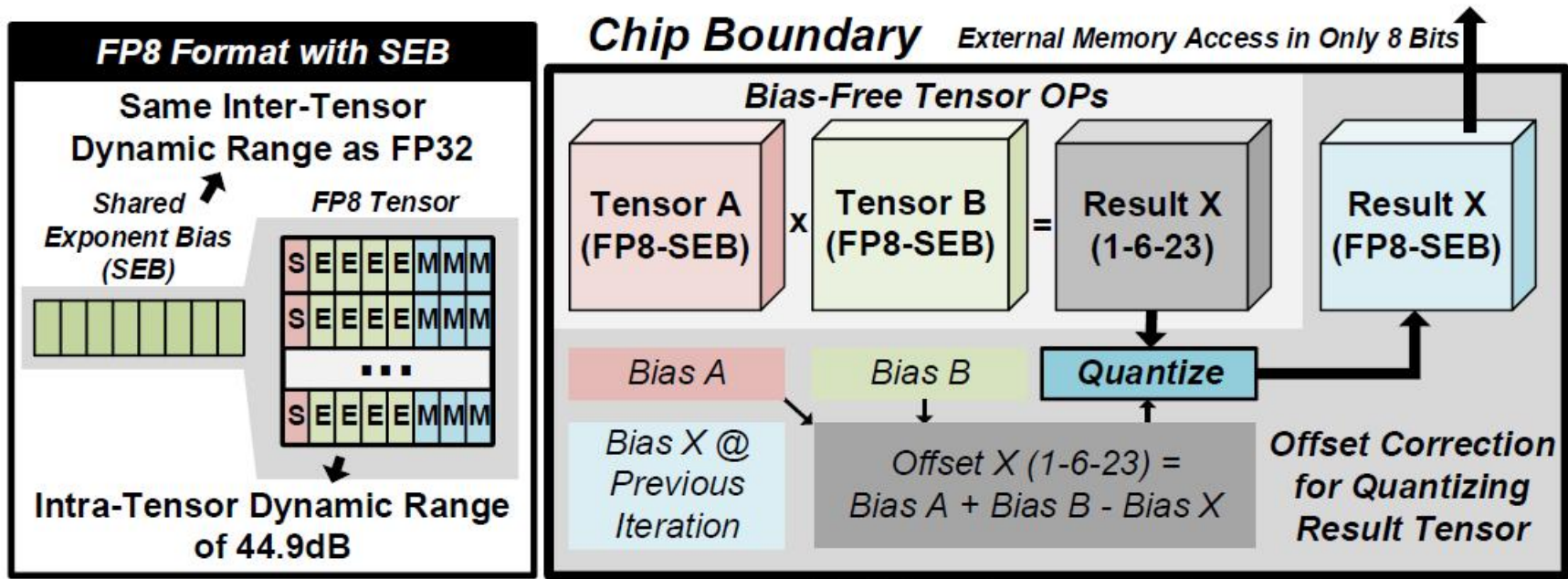


Fig. 8. Statistic Quantization Unit. ( $X$ : unquantized data;  $\Theta$ : quantization parameters;  $X_q$ : quantized data.)

# / Training Accelerator

- 解决方案4: 共享指数位, 增加表示范围

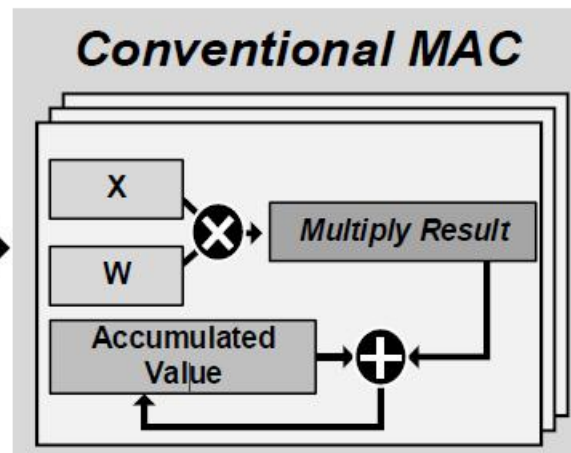
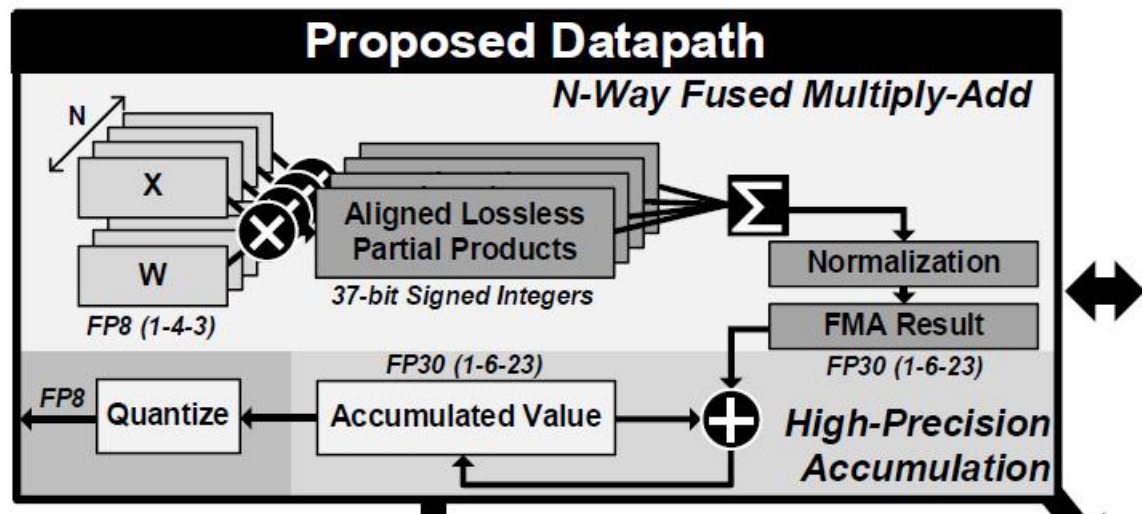
ISSCC@2021



# / Training Accelerator

- 解决方案4：共享指数位，增加表示范围
- 计算核心的修改：

ISSCC@2021

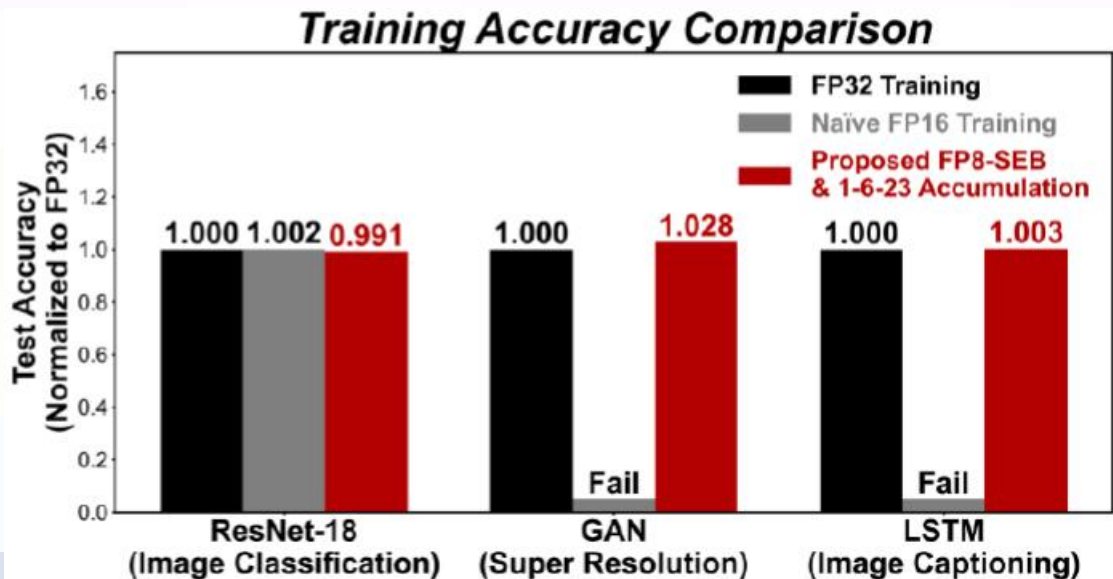


# / Training Accelerator

- 解决方案4：共享指数位，增加表示范围

效果：

ISSCC@2021



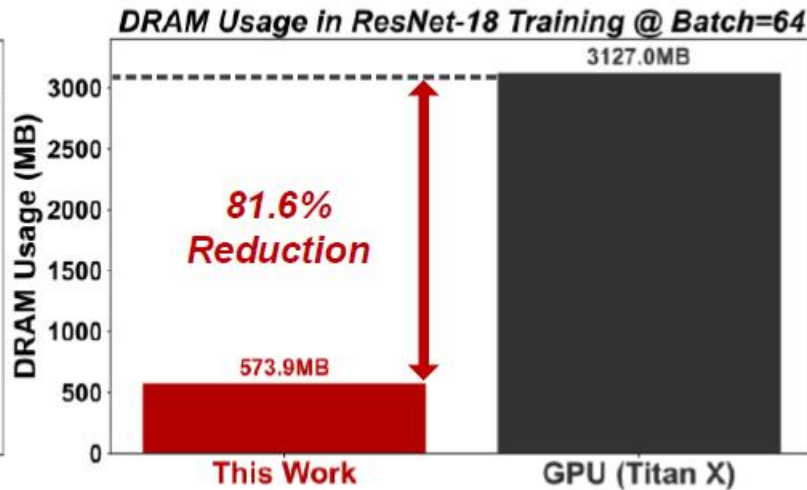
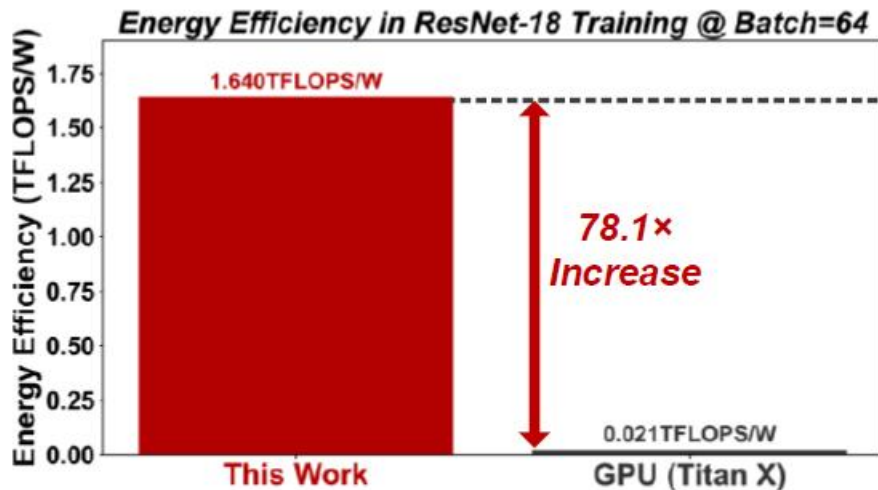
- 可以看到，如果不采取一定的优化办法，FP16是训不出某些网络的（GAN、LSTM）等。

# / Training Accelerator

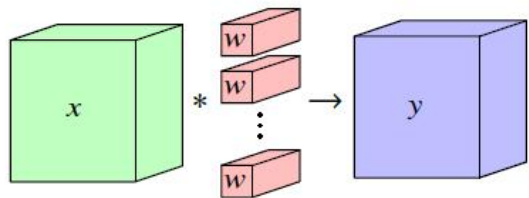
- 解决方案4: 共享指数位, 增加表示范围

ISSCC@2021

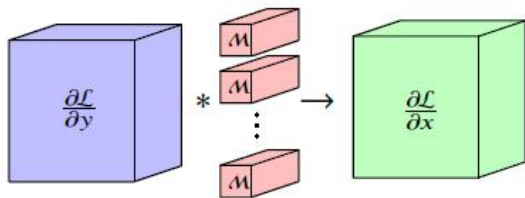
大幅提升能效:



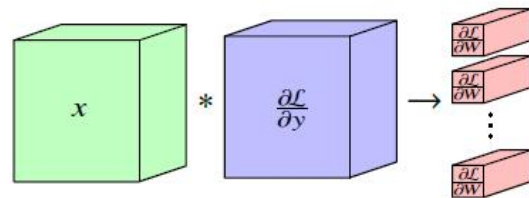
# / Training Accelerator



(a) forward:  $x * W \rightarrow y$



(b) backward:  $\frac{\partial \mathcal{L}}{\partial y} * W^\top \rightarrow \frac{\partial \mathcal{L}}{\partial x}$



(c) weight update:  $x * \frac{\partial \mathcal{L}}{\partial y} \rightarrow \frac{\partial \mathcal{L}}{\partial W}$

- 从推理加速器中获取灵感
  - 稀疏？前向推理有ReLU函数制造稀疏，反向传播由于batchnorm层的存在，梯度张量基本不存在稀疏性。



## / Training Accelerator

- 训练加速器，前向和反向需要做到一块芯片上。

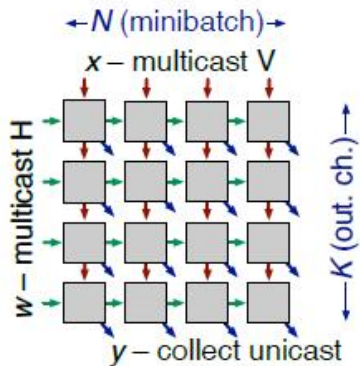
|    | Weight | Activation | Gradient |
|----|--------|------------|----------|
| 前向 | 稀疏     | 稀疏         | ---      |
| 反向 | 稀疏     | 稀疏         | 不稀疏      |

- 如果利用上节课讲的稀疏计算加速器，会带来什么问题？
  - 前向可加速，反向不能加速，加速效果折半
  - 稀疏加速器有复杂的控制硬件开销，这一切可能就不再值得。
  - 但仍可以考虑权重、激活各自的稀疏性



# /Training Accelerator

MICRO@2020



|              | fw  | bw                                  | wu                                  |
|--------------|-----|-------------------------------------|-------------------------------------|
| (H)orizontal | $w$ | $w$                                 | $\partial \mathcal{L} / \partial w$ |
| (V)ertical   | $x$ | $\partial \mathcal{L} / \partial x$ | $x$                                 |
| (U)nicast    | $y$ | $\partial \mathcal{L} / \partial y$ | $\partial \mathcal{L} / \partial y$ |

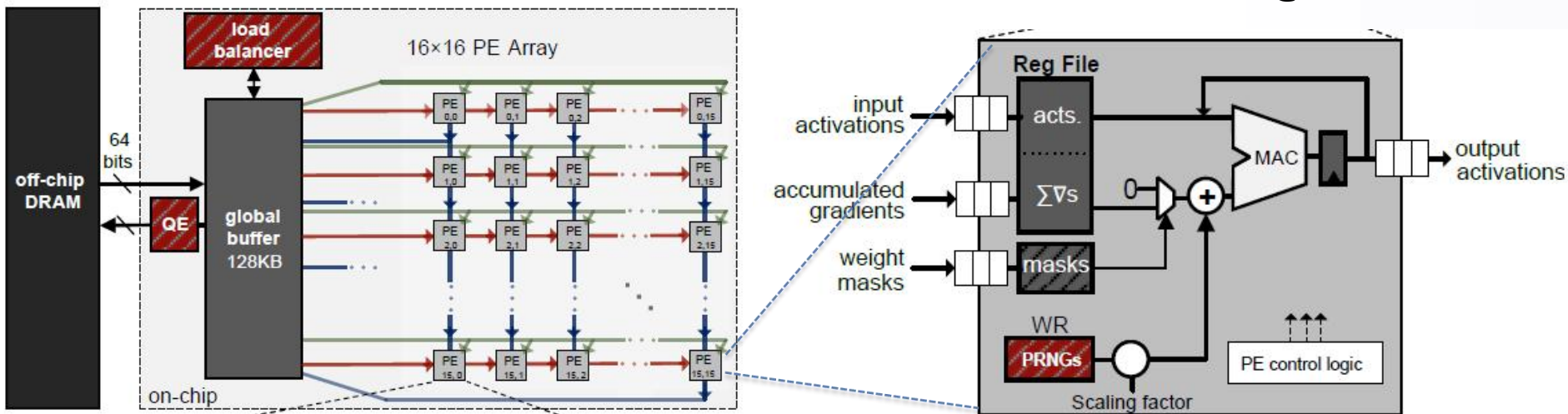
$w$  weights       $x$  input activations  
 $y$  output activation partial sums

计算核心设计

在一个计算核心上做三种计算模式

# / Training Accelerator

MICRO@2020



架构设计

对Weight做CSC编码，利用weight的稀疏性

# / Training Accelerator

- 梯度真的不存在稀疏吗？
- 回顾权重剪枝产生稀疏的策略。
  - 舍小放大

# / Training Accelerator

DAC@2020

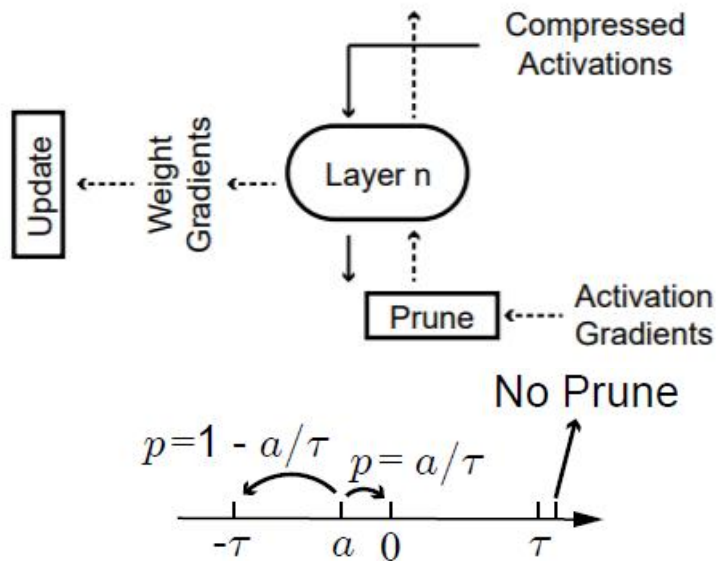
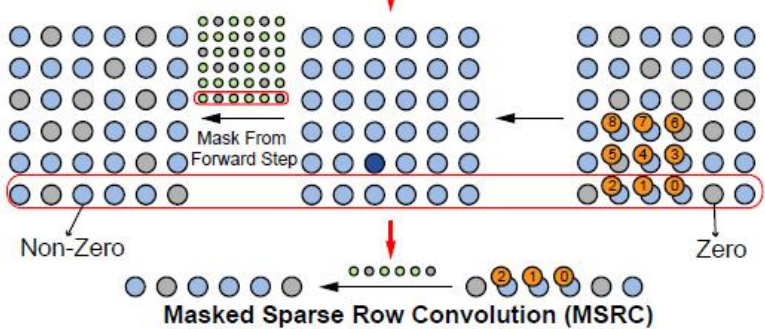
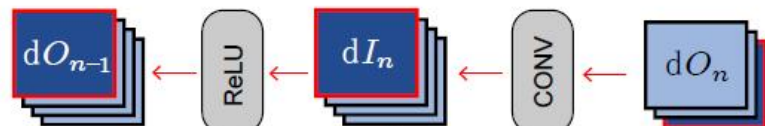
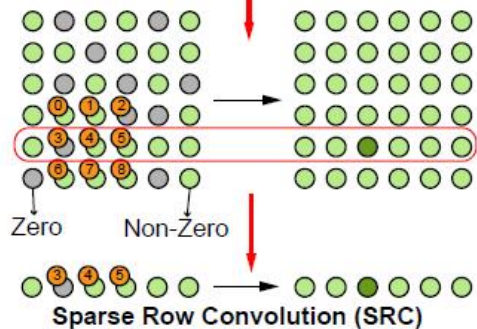
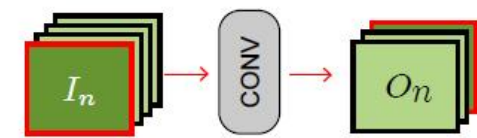


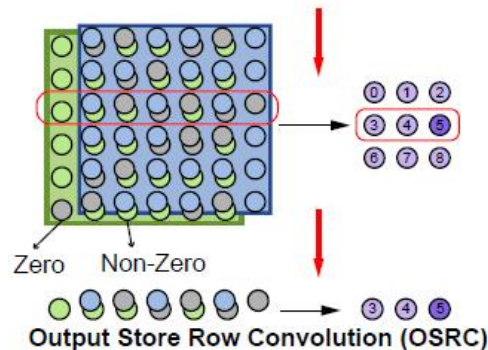
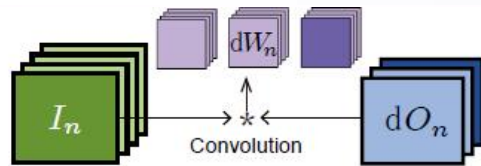
Fig. 3: Stochastic gradients pruning algorithm.  $\tau$  is the threshold that determines whether a value should be pruned.  $p$  is the probability of setting a value to 0 or  $\pm\tau$ .

- 在线产生稀疏
- 依概率将小梯度归为0或阈值。

# / Training Accelerator

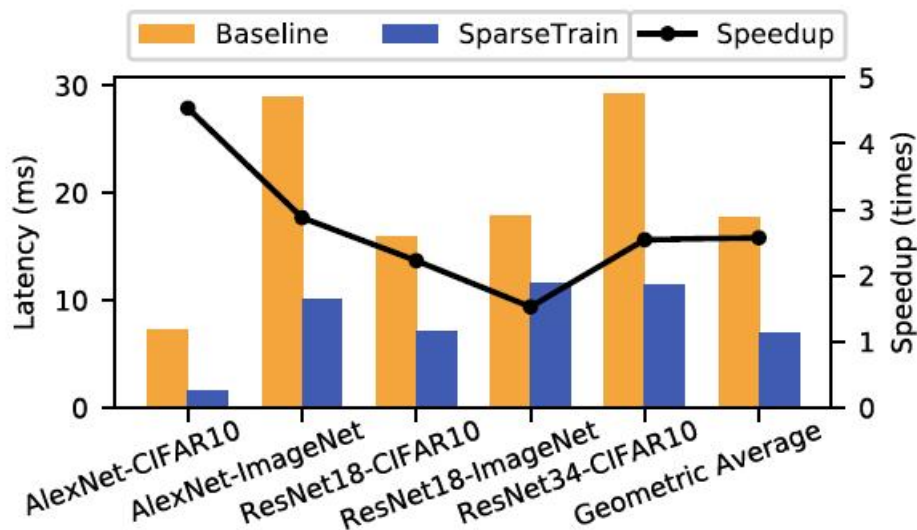


DAC@2020

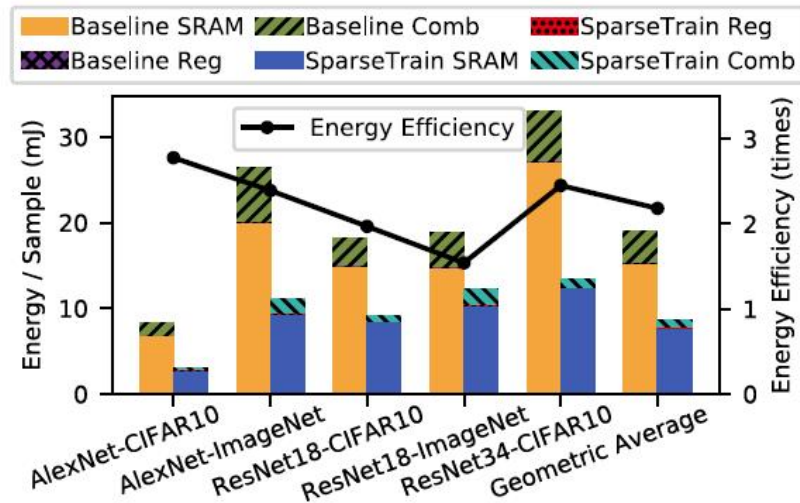


- 1D PE阵列，做mask sparse row convolution

# / Training Accelerator



DAC@2020

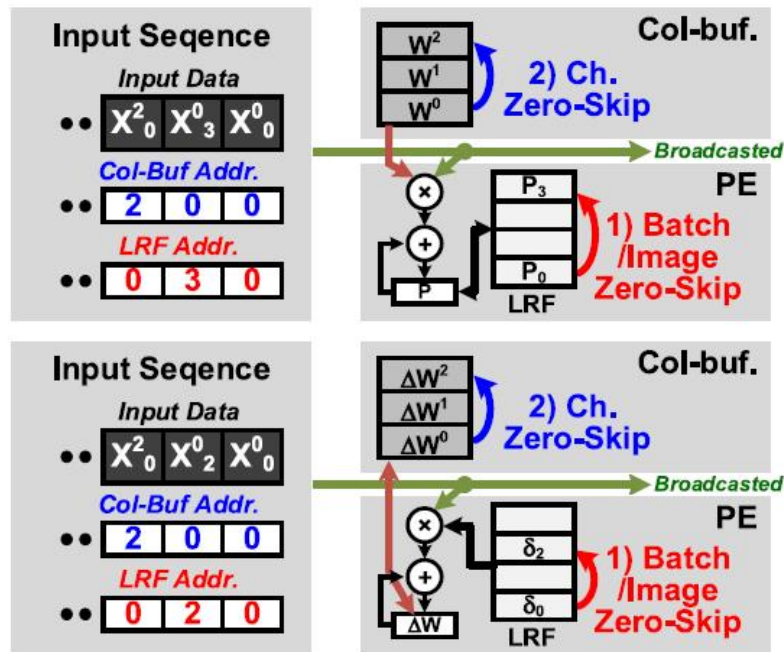
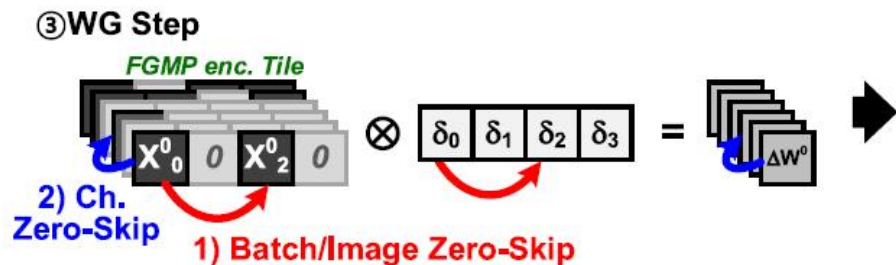
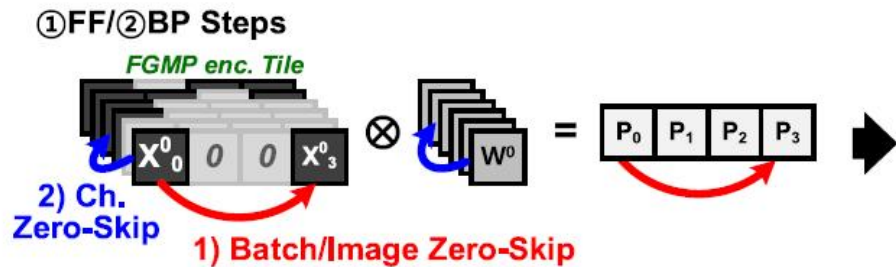


速度更快，能效更好

# / Training Accelerator

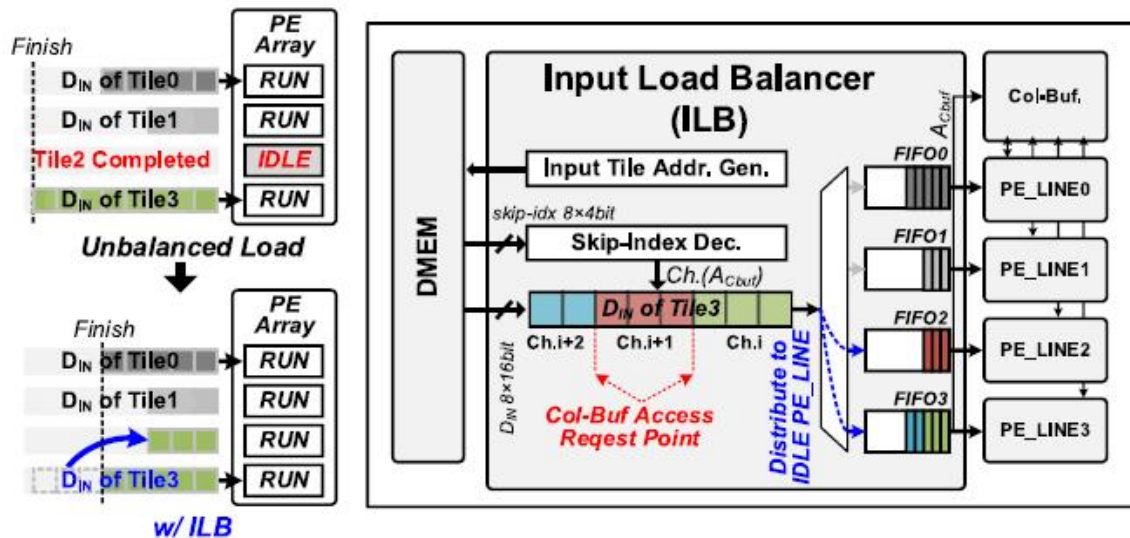
ISCL@2019

FF: 前向传播, BP: 反向传播, WG: 梯度更新



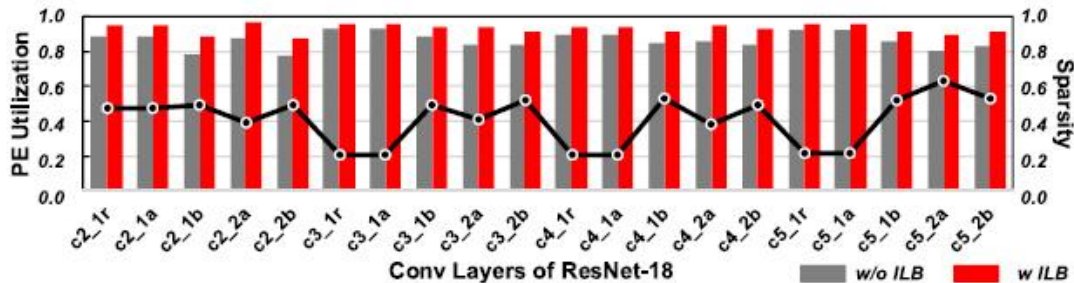


# Training Accelerator



ISCL@2019

负载平衡设计



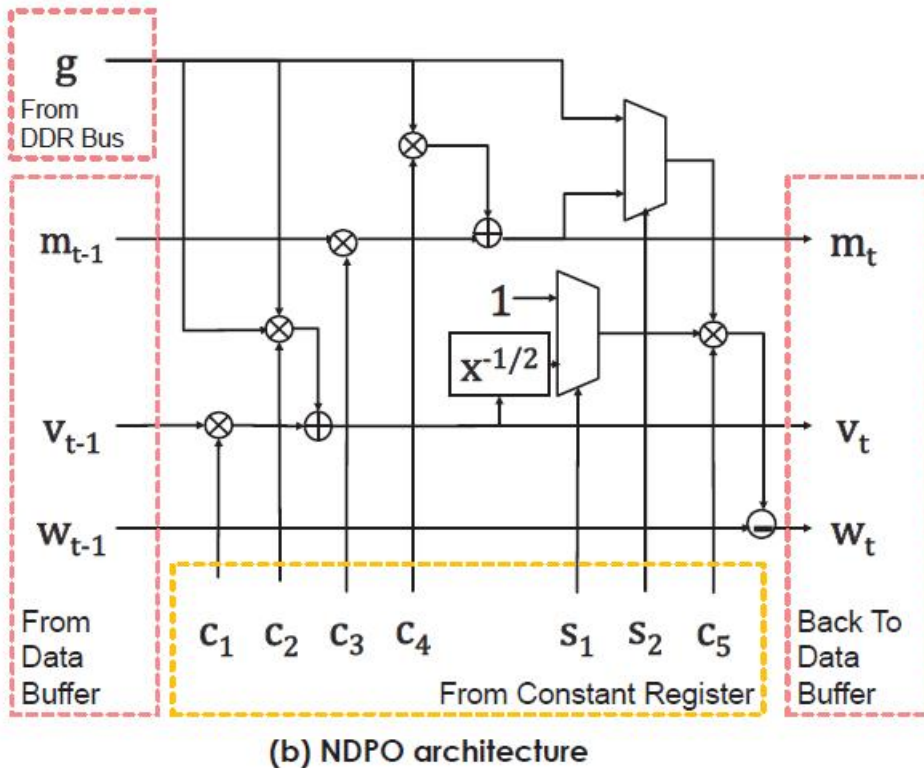
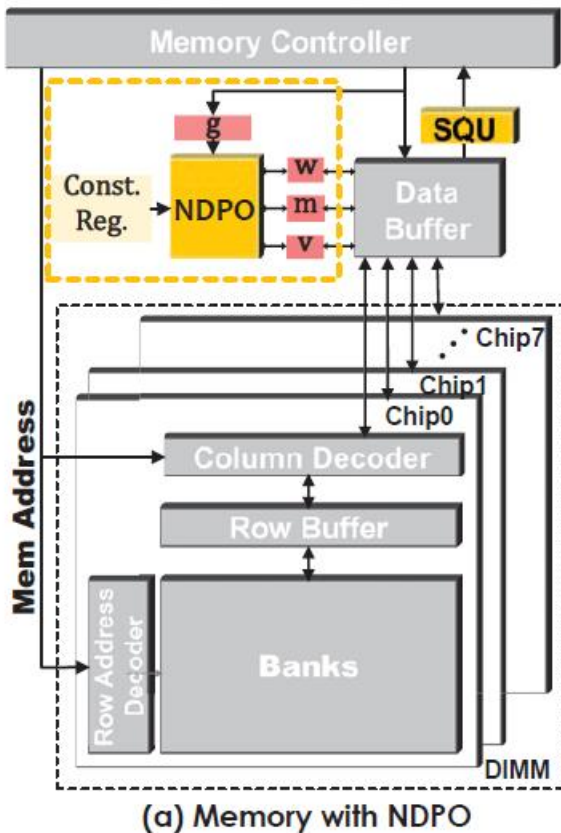


# /Training Accelerator

- 还有其他设计思路吗？
- 更新参数、传递梯度都需要反复读取权重。

——near data processing philosophy

NDPO: near data processing optimizer



# / Training Accelerator

ISCA2021

| Optimizer    | Parameters Maintained | Computations Performed                                                                                                                                                                                                                                                        |
|--------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SGD          |                       | $w_t = w_{t-1} - \eta \times g$                                                                                                                                                                                                                                               |
| AdaGrad [15] | $m$                   | $m_t = m_{t-1} + g^2$<br>$w_t = w_{t-1} - \eta \times g \times m_t^{-\frac{1}{2}}$                                                                                                                                                                                            |
| RMSProp [25] | $m$                   | $m_t = \beta \times m_{t-1} + (1 - \beta) \times g^2$<br>$w_t = w_{t-1} - \eta \times g \times m_t^{-\frac{1}{2}}$                                                                                                                                                            |
| Adam [36]    | $m, v$                | $m_t = \beta_1 \times m_{t-1} + (1 - \beta_1) \times g$<br>$v_t = \beta_2 \times v_{t-1} + (1 - \beta_2) \times g^2$<br>$\hat{m}_t = m_t / (1 - \beta_1^t)$<br>$\hat{v}_t = v_t / (1 - \beta_2^t)$<br>$w_t = w_{t-1} - \eta \times \hat{m}_t \times \hat{v}_t^{-\frac{1}{2}}$ |

该架构在DRAM附近

做权重更新，支持多

种优化器，降低数据

传输消耗

# Review

- **Training Flow**
  - **Large training time**
  - **Use better training scheme**
- **Optimizer**
  - **Find better direction**
  - **Adaptive step length**
- **Training Accelerator**
  - **Quantization**
  - **Sparse Training**